



Estrutura de Dados I - USP São Carlos

Projeto 1

Polinômios Esparsos

Prof. Dra. Elaine Parros Machado de Sousa

Maria Eduarda Aissa Andrade - 17911140 - mariaeduarda.aissa@usp.br

Sofia Chen Ying Qi - 17973244 - sofiacyq@usp.br

Vivian Machado Costa - 17927892 - vivian.machado@usp.br

Índice

1	Projeto da Solução	3
1.1	O problema inicial e a estrutura escolhida	3
1.2	Implementação das operações	3
2	Justificativa	4
3	Medição	5
3.1	Resultados da medição	5
4	Casos de teste	6
4.1	Caso 1	6
4.2	Caso 2	6
4.3	Caso 3	7
4.4	Caso 4	7
5	Ponto fraco	7
6	Referências Bibliográficas	7
	Apêndice A Tabela de Custos	8

1 Projeto da Solução

1.1 O problema inicial e a estrutura escolhida

Polinômios esparsos são polinômios que possuem muitos coeficientes iguais a zero. Ao invés de armazenar todos os termos, armazena-se apenas os coeficientes não nulos e seus respectivos expoentes, economizando memória e tempo de processamento. Um exemplo de polinômio esparsos é o polinômio $P(x) = 10x^{100} - 5x^2 + 2$, com grau 100, porém apenas três termos.

Necessita-se, dessa forma, armazenar todos os elementos não nulos e escolher como será feito, da melhor maneira. Assim, escolheu-se como estrutura a lista encadeada dinâmica ordenada.

1.2 Implementação das operações

A estrutura de dados presente em "tadelista.h" possui:

- Termo: guarda um par (coeficiente, grau), ambos do tipo long long - permitindo o armazenamento de coeficientes e graus grandes;
- Nó: representa um nó da lista, contém um termo e um ponteiro (prox) para o próximo nó;
- Polinomio: ponteiro "inicio" para o primeiro nó.

Com isso, pode-se analisar o módulo de operações sobre os polinômios "tadelista.c". Temos que:

- "criapolinomio" aloca a struct Polinomio e inicializa: "inicio = NULL" (lista vazia);

- "removepoli(poli, Grau)" busca o nó de grau exato pela mesma varredura ordenada; se não encontrar (ou a lista terminar antes), não faz nada; se encontrar, desconecta o nó da lista, ajustando "inicio" ou "anterior→prox" e libera sua memória;
- "removemenor" percorre até o fim guardando o penúltimo nó em "anterior", desconecta o último e libera. Isso é possível pela ordenação da lista. A função também trata o caso de lista com um único elemento (essa se torna vazia);
- "liberapoli" percorre a lista nó a nó, liberando cada um. Guarda o próximo nó antes de liberar o atual, para não perder a referência, e por fim libera a própria struct "Polinomio";
- "coeficiente(poli, Grau)" percorre a lista ordenada até achar o grau ou passar dele. Assim, pode retornar o coeficiente - caso o encontre -, ou 0 (tanto para "não existe" quanto "polinômio nulo ou vazio");
- "graumax" retorna o grau do primeiro nó, já que a lista está ordenada decrescentemente. Retorna -1 para polinômio nulo ou lista vazia;
- "soma(A, B)" cria um polinômio vazio e insere, um a um, todos os termos de A, e depois todos os termos de B usando "inserepoli". Como "inserepoli" já soma coeficientes de mesmo grau automaticamente, não há, necessariamente, uma lógica de soma dedicada. Além disso, a função possui verificação para não desreferenciar

A ou B caso sejam nulos.

- "multiplica(A, B)" implementa uma distributiva em que, para cada termo de A, percorre todos os termos de B calculando um novo coeficiente ($\text{NovoCoef} = \text{coefA} * \text{coefB}$) e um novo grau ($\text{NovoGrau} = \text{grauA} + \text{grauB}$), inserindo cada resultado no polinômio de saída via "inserepoli";
- "escala(poli, C)" percorre a lista multiplicando cada coeficiente por C diretamente para C diferente de 0. Caso contrário, libera todos os nós e esvazia a lista - tratando, dessa forma, o caso especial em que C é igual à zero;
- "imprime_poli" percorre a lista do início ao fim, imprimindo cada termo no formato $\text{coeficiente} * x^{\text{grau}}$, controlando os espaços entre termos com uma flag "prim". Se a lista estiver vazia, imprime 0;
- "imprime_inverso" faz a impressão em ordem crescente de grau. Troca o "prox" de cada nó para inverter a lista, imprime na nova ordem, e depois inverte novamente para restaurar a lista ao estado original antes de retornar.

Pode-se dar destaque à função "inserepoli" com os parâmetros "poli", "coef" e "grau", respectivamente. Essa é a função central, em que se o coeficiente for igual à 0, mantém a invariante de que nenhum termo com coeficiente zero existe na lista. Ademais, trata a lista vazia inserindo um novo nó diretamente como "inicio", e trata a inserção na cabeça quando o grau do novo termo é maior que o do primeiro nó;

caso contrário, percorre a lista com dois ponteiros (atual e anterior) enquanto o grau atual for maior que o grau buscado - procura o ponto de inserção mantendo a ordem decrescente.

Caso a função "inserepoli" encontre um nó de mesmo grau, essa soma os coeficientes e, se a soma zerar, remove o nó da lista. Se não houver grau igual, insere o novo nó entre "anterior" e "atual".

2 Justificativa

A escolha da estrutura utilizada foi baseada principalmente em dois critérios: a eficiência assintótica de operações frequentes no programa e a simplicidade e reaproveitamento de código proporcionados pela estrutura. Uma lista encadeada dinâmica ordenada beneficia diretamente as chamadas de "graumax" e "imprime_poli", correspondentes aos comandos "GRAU" e "IMPRIME". A ordenação decrescente favorece esses comandos por eliminar a necessidade de comparações de grau para localizar o termo desejado: no comando "GRAU", como o maior grau está sempre na cabeça da lista, a operação é $\mathcal{O}(1)$; e, além disso, na impressão canônica, a lista já está na ordem correta, dispensando qualquer reorganização antes de imprimir.

Além disso, a dinamicidade evita o desperdício de memória associado à sobrealocação típica de arrays com crescimento por dobramento, embora isso tenha como contrapartida a sobrecarga de armazenar um ponteiro adicional (prox) em cada nó - um "trade-off" de espaço, não uma vantagem unilateral. Raciocínio análogo foi aplicado ao array dinâmico usado na ta-

bela de polinômios, em que a ordem dos nomes não importa, já que o seu trabalho é apenas associar nomes a polinômios para os comandos, então a escolha de um array dinâmico foi direcionada ao critério de armazenamento flexível, permitindo que a capacidade cresça conforme necessário sem um limite fixo.

Nesse contexto, o uso de uma lista ordenada se mostra conveniente pelo reaproveitamento da função "inserepoli" em outras funções. Como essa função mantém a lista ordenada e centraliza o mecanismo de soma de coeficientes de termos com grau igual, removendo automaticamente o nó caso o resultado seja zero, o programa simplifica e reduz a duplicação lógica entre os comandos, mesmo que isso implique em uma perda de desempenho em algumas operações específicas.

A tabela de custos, disponível no Apêndice A expõe que a lista ordenada simplesmente ligada estática possui os mesmos custos da lista implementada, vale destacar, portanto, que optou-se pela flexibilidade da memória. Além disso, a análise comparando operações isoladas pode indicar uma eficiência maior no uso do vetor ordenado. Entretanto, escolheu-se a lista encadeada ordenada dinâmica pela análise das operações que o programa realmente executa, e suas frequências. É possível observar, por exemplo, que a função de inserção é $\mathcal{O}(n)$ tanto na lista quanto no vetor - mesmo com a busca binária, a vantagem do vetor não se propaga para a inserção. Dessa forma, é possível dizer que as vantagens do vetor aparecem em operações pontuais para o problema inicial de inserções, sendo assim, reconhece-se ex-

plicitamente que se o programa tivesse um perfil de uso diferente, com muitas chamadas de "REMOVEMENOR" por exemplo, o vetor ordenado seria a escolha assintoticamente superior. Portanto, a estrutura foi escolhida, para além dos argumentos expostos acima, porque analisou-se um padrão de uso de inserções mais recorrente.

3 Medição

A medição do tempo utilizado pelo código escrito para efetuar os comandos necessários carece de uma entrada significativa. Dessa forma, produziu-se, também, um gerador de dois polinômios, a fim de medir o tempo necessário para execução de comandos entre estes.

A partir do gerador, pode-se imprimir a entrada teste necessária para a medição, com dois polinômios principais, cada um com 50.000 termos de graus sorteados de forma única. Além disso, imprime-se uma sequência de comandos sugeridos para simular a carga de processamento sobre a lista encadeada (IMPRIME, GRAU e SOMA).

3.1 Resultados da medição

Assim, pode-se obter os dados expostos na Tabela 1.

Tabela 1: Resultados obtidos com o gerador.

Métrica	Resultado
Carga de Dados	100000 nós alocados
Tempo total (Segundos)	6,78s
Tempo total (Milissegundos)	6781,34ms

É possível dizer que as operações sugeridas favorecem o tipo de estrutura escolhida.

Como discutido anteriormente, a estrutura propicia a impressão dos polinômios de forma eficiente e direta $\mathcal{O}(N)$ - a ordenação dos termos anterior à exibição custaria $\mathcal{O}(N \log N)$ ou $\mathcal{O}(N^2)$. Outrossim, o grau de maior valor estará de forma acessível no início da lista, constituindo, então, o acesso ao grau do polinômio uma operação em tempo constante $\mathcal{O}(1)$, bem como a soma entre os polinômios poderá ser realizada de forma altamente efetiva devido a essa ordenação.

Dessa forma, a estrutura escolhida provou-se capaz de mitigar o tempo de operações frequentes, como GRAU ($\mathcal{O}(1)$), SOMA ($\mathcal{O}(N_A + N_B)$) e a própria exibição dos termos. Em suma, o tempo de 6,78 segundos valida a estabilidade da implementação sob cenários estressantes de carga, demonstrando que a escolha das estruturas de dados e a manutenção da ordenação prévia dos termos alcançaram um balanço computacional adequado.

4 Casos de teste

Além da medição do desempenho do programa, necessita-se testar casos específicos que poderiam se tornar problemas na aplicabilidade desse. Dessa forma, foram criados 4 casos testes principais.

4.1 Caso 1

O primeiro caso trata da definição de um polinômio com graus repetidos, e, além disso, da remoção de um termo em que o coeficiente é zerado. O programa receberá dois polinômios, A e B, sendo "A" o polinômio com graus repetidos, e a soma desses polinômios resulta em um terceiro

sem o termo de grau 3, exposto na Figura 1.

```
DEF A 4
1 5
2 3
3 3
5 1
DEF B 3
3 2
-5 3
4 0
IMPRIME A
1*x^5 5*x^3 5*x^1
IMPRIME B
-5*x^3 3*x^2 4*x^0
SOMA A B A
IMPRIME A
1*x^5 3*x^2 5*x^1 4*x^0
FIM
```

Figura 1: Caso 1

4.2 Caso 2

Com o primeiro caso concluído com êxito, pôde-se seguir para o segundo, mostrado na Figura 2. A fim de testar diferentes operações com um polinômio nulo, é definido este com 0, e pede-se para o programa realizar diferentes operações com esse.

```
DEF A 0
IMPRIME A
0
GRAU A
-1
COEF A 3
0
REMOVE A 5
IMPRIME A
0
ADD A 0 3
IMPRIME A
0
REMOVEMENOR A
IMPRIME A
0
FIM
```

Figura 2: Caso 2

Ao definir o polinômio "A" como nulo, o programa retorna -1 para o grau deste, 0 para o coeficiente de grau ausente no polinômio, e, além disso, a indiferença de operações como "REMOVE", "ADD" com

coeficiente 0, e "REMOVEMENOR" de polinômio nulo.

4.3 Caso 3

O terceiro caso trata da adição de um termo com grau já existente no polinômio, sendo assim, os coeficientes somados - bem como em DEF -, da redefinição de um polinômio, e o uso de "ESCALA" com "c" (inteiro que multiplica todos os coeficientes do polinômio) igual à zero. O êxito do teste pode ser observado na figura 3.

```
DEF A 1
3 2
ADD A 2 2
IMPRIME A
5*x^2
DEF A 1
5 0
IMPRIME A
5*x^0
ESCALA A 0
IMPRIME A
0
FIM
```

Figura 3: Caso 3

4.4 Caso 4

O último caso expõe a validade do uso de "LIBERA" logo após a definição de um polinômio, como mostra a Figura 4

```
DEF A 2
2 2
1 3
4 0
LIBERA A
FIM
```

Figura 4: Caso 4

5 Ponto fraco

Os pontos fracos da implementação aparecem em polinômios com muitos termos, em especial quando exigida a remoção ou busca do termo de menor grau. Por exemplo, a operação "REMOVEMENOR" percorre

toda a lista até encontrar o penúltimo nó, gerando um custo $\mathcal{O}(n)$, que se repete a cada chamada. Um exemplo que exhibe esse problema é a operação "DEF" com "n" termos seguido por "n" chamadas de "REMOVEMENOR" até que se tenha o polinômio inteiro vazio, isso totalizaria um custo de $\mathcal{O}(n^2)$, ou seja, esse tipo de caso se torna muito custoso para a implementação.

Ademais, a busca por um coeficiente específico (COEF), sem acesso indexado, não possibilita realizar busca binária, gerando assim um custo $\mathcal{O}(n)$, assim, muitas buscas em um polinômio grande não são eficientes.

Além disso, a estrutura se mostra menos eficaz do que outras nas funções de soma e multiplicação, pois essas operações reutilizam a função de inserção, e, desse modo, a cada chamada a função percorre a lista toda até achar a posição correta em que aqueles novos termos gerados dentro da soma/multiplicação se encontram, assim tendo que encaixar termo a termo em seu devido lugar, ou seja, gerando um custo de $\mathcal{O}(n \times m)$ no pior caso, o que é mais custoso em relação a outras estruturas que poderiam gerar um custo de $\mathcal{O}(n + m)$.

Logo, a estrutura escolhida mostra vantagens em pontos como um algoritmo simples, mas gera um custo em funções mais exigentes.

6 Referências Bibliográficas

Aula 2 - Análise de Algoritmos - Parte 2. 2026. 21 slides. Apresentação de slides. Disponível em: <https://edisciplinas.usp.br>. Acesso em: 3 set. 2026. Circulação res-

trita.

Aula 3 - TAD. 2026. 37 slides. Apresentação de slides. Disponível em: <https://edisciplinas.usp.br>. Acesso em: 3 set. 2026. Circulação restrita.

Aula 4 - Listas Sequenciais. 2026. 31 slides. Apresentação de slides. Disponível em: <https://edisciplinas.usp.br>. Acesso em: 4 set. 2026. Circulação restrita.

Aula 5 - Listas Encadeadas Estáticas. 2026. 26 slides. Apresentação de slides. Disponível em: <https://edisciplinas.usp.br>. Acesso em: 4 set. 2026. Circulação restrita.

Aula 6 - Listas Encadeadas Dinâmicas. 2026. 18 slides. Apresentação de slides. Disponível em: <https://edisciplinas.usp.br>. Acesso em: 7 set. 2026. Circulação restrita.

FEOFILOFF, Paulo. Listas encadeadas. IME, 2018. Disponível em: <https://www.ime.usp.br/~pf/algoritmos/aulas/lista.html>. Acesso em: 7 set. 2026.

A Tabela de Custos

Tabela A1: Comparação de custos assintóticos entre estruturas consideradas

Operação	Lista encadeada dinâmica ordenada (implementação)	Vetor ordenado (dinâmico)	Lista ordenada simplesmente ligada (estática)	Lista encadeada dinâmica não ordenada
Busca(L , grau)	$O(n)$	$O(\log n)$	$O(n)$	$O(n)$
Insere(L , coef, grau)	$O(n)$	$O(n)^*$	$O(n)$	$O(n)^{**}$
Remove(L , grau)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
RemoveMenor(L)	$O(n)$	$O(1)$	$O(n)$	$O(n)$
GrauMax(L)	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Imprime(L)	$O(n)$	$O(n)$	$O(n)$	$O(n \log n)^{***}$

* Busca binária localiza a posição em $O(\log n)$, mas o deslocamento dos elementos posteriores para abrir espaço custa $O(n)$, dominando o total.

** Mesmo sem ordenação, é preciso varrer a lista inteira para checar se já existe termo de mesmo grau (regra de soma de coeficientes iguais).

*** Impressão em ordem canônica decrescente exigiria ordenar a lista antes de imprimir.